

Competitive Programming

Section 62 Editorial: Codeforces Course Ladder

Competitive Programming Course

2026-07-10

Table of contents

1	How to use these editorials	4
2	1. 71A – Way Too Long Words	5
2.1	Restatement and recognition	5
2.2	Algorithm, proof, and tests	5
3	2. 4C – Registration System	6
3.1	State meaning	6
3.2	Why no extra occupied-name set is needed	6
4	3. 363B – Fence	7
4.1	Observation	7
4.2	Algorithm and proof	7
5	4. 279B – Books	8
5.1	Invariant	8
5.2	Algorithm, proof, and complexity	8
6	5. 158B – Taxi	9
6.1	Forced greedy packing	9
6.2	Proof route	9
7	6. 706B – Interesting Drink	10
7.1	Recognition and algorithm	10
7.2	Correctness and traps	10
8	7. 520B – Two Buttons	11
8.1	State graph	11
8.2	Algorithm and proof	11
9	8. 455A – Boredom	12
9.1	Compress the story by value	12
9.2	Correctness and implementation	12
10	9. 300C – Beautiful Numbers	13
10.1	Count by number of a digits	13
10.2	Proof and complexity	13
11	10. 166B – Polygons	14
11.1	Reduce containment to convex point location	14
11.2	Why this is sufficient	14
12	11. 20C – Dijkstra?	15
12.1	Model and algorithm	15
12.2	Correctness proof	15
13	12. 161D – Distance in Tree	16
13.1	Bounded-depth tree DP	16

13.2 Merge, proof, and complexity	16
14 13. 580D – Kefa and Dishes	17
14.1 State captures the only relevant history	17
14.2 Proof and costs	17
15 14. 1245D – Shichikuji and Power Grid	18
15.1 Virtual-source reduction	18
15.2 Algorithm and proof	18
16 15. 271D – Good Substrings	19
16.1 Enumerate starts, identify contents exactly	19
16.2 Correctness and testing	19
17 16. 61E – Enemy Is Weak	20
17.1 Split each decreasing triple at its middle	20
17.2 Proof and complexity	20
18 17. 86D – Powerful Array	21
18.1 Maintain the statistic while moving a range	21
18.2 Why it works	21
19 18. 498C – Array and Operations	22
19.1 Decompose operations by prime	22
19.2 Reduction proof	22
20 19. 319C – Kalila and Dimna in the Logging Industry	23
20.1 Derive lines before choosing a hull	23
20.2 Hull invariant and proof	23
21 Stress and carry-forward	24

1 How to use these editorials

Each block is an original derivation, not a substitute for attempting the linked task. Read the restatement only to compare your interpretation. Then look for the recognition signal, reproduce the proof route, and finally check the implementation traps. For external problems, the course teaches the tools; these notes teach why that particular statement calls for them.

2 1. 71A – Way Too Long Words

Official: <https://codeforces.com/problemset/problem/71/A>

2.1 Restatement and recognition

Transform each independent word according to one length threshold. There is no interaction between cases, so the correct model is a direct input/output loop, not a string algorithm.

2.2 Algorithm, proof, and tests

For a word `s`, leave it unchanged when `len(s) <= 10`; otherwise output `s[0]`, `len(s)-2`, and `s.back()`. The two branches partition all legal word lengths. In the long branch, exactly the first and last characters are kept, so the count between them is exactly `len(s)-2`; therefore the output matches the requested abbreviation. The time is $O(\text{total input characters})$.

Test lengths 1, 10, 11, and 100. In C++, concatenate `string` and `to_string`; in Python use an f-string. Do not treat the threshold as `>= 10`.

3 2. 4C – Registration System

Official: <https://codeforces.com/problemset/problem/4/C>

3.1 State meaning

Maintain `next[name]`: the smallest suffix not yet suggested for this base name, with zero meaning that the unsuffixed name has never appeared. On a new base name, print `OK` and set `next[name] = 1`. Otherwise print `name + next[name]` and increment it.

3.2 Why no extra occupied-name set is needed

The requests themselves are lowercase base names, while every generated suffix for one base is issued in increasing order. Once suffix `t` is printed, all smaller positive suffixes for that base were printed earlier, and `t+1` is the next free one. The map invariant therefore yields the smallest allowed suffix. Expected time is $O(\text{total name length})$ with a hash map, and memory is linear in the distinct requested names.

Test the same base three times, two interleaved bases, and a request that looks like another generated name. Use `unordered_map<string,int>` or `dict`; never scan suffixes from 1 on every duplicate.

4 3. 363B – Fence

Official: <https://codeforces.com/problemset/problem/363/B>

4.1 Observation

Every candidate is a fixed-length window. After computing the first sum, moving the window one step removes exactly its leftmost height and adds exactly its new rightmost height.

4.2 Algorithm and proof

Set $\text{sum} = a[0] + \dots + a[k-1]$, remember it as the current best, and for each right endpoint $r \geq k$ update $\text{sum} += a[r] - a[r-k]$. Compare the resulting window $[r-k+1, r]$ to the best. By induction, just before each comparison sum equals the sum of that window: it is true initially, and the update removes/adds precisely the changed elements. Every length- k window appears once, so the minimum remembered sum is globally minimum. Time $O(n)$, memory $O(1)$ besides input.

Output the stored left endpoint plus one. Test $k=1$, $k=n$, and equal window sums; the problem accepts any minimizer, but your comparison decides the tie.

5 4. 279B – Books

Official: <https://codeforces.com/problemset/problem/279/B>

5.1 Invariant

All reading times are positive. Keep a window $[l, r]$ with sum at most t after repeatedly advancing l while it is too expensive. Then it is the longest feasible window ending at r : moving l farther right only shortens it, and moving it left would violate the budget.

5.2 Algorithm, proof, and complexity

Extend r from left to right and add $a[r]$. While the sum exceeds t , remove $a[l++]$. Maximize $r-l+1$. The invariant above proves each recorded length is feasible and no feasible window ending at r is longer. Each endpoint advances at most n times, yielding $O(n)$ time and $O(1)$ extra memory.

The positivity is essential: with negative times, a removed prefix could later become useful and this proof fails. Test a single over-budget book, an exact budget, and an optimal window beginning after index zero.

6 5. 158B – Taxi

Official: <https://codeforces.com/problemset/problem/158/B>

6.1 Forced greedy packing

Count groups of sizes 1, 2, 3, 4. Every size-4 group is alone. Every size-3 group must occupy a taxi and can take at most one size-1 group, so pair those first. Pair size-2 groups together; one remaining size-2 can take up to two ones. Finally pack remaining ones four per taxi.

6.2 Proof route

The size-3 decision is forced: no group other than a one can share its taxi, so using a one there cannot hurt a future fuller packing. Size-2 groups only fit with another size-2 or ones; pairing twos maximizes the remaining capacity for ones. After those forced classes, only ones remain, and four per taxi is optimal by capacity. Thus the construction uses the minimum possible taxis.

All operations are $O(n)$. Test no ones for size-3 groups, an odd number of twos, and one/two/three/four leftover ones. Do not simulate individual taxis.

7 6. 706B – Interesting Drink

Official: <https://codeforces.com/problemset/problem/706/B>

7.1 Recognition and algorithm

The same static set of prices serves many threshold queries. Sort it once. For a budget x , the answer is the first index strictly greater than x , namely `upper_bound(prices.begin(), prices.end(), x) - prices.begin()` in C++ or `bisect_right(prices, x)` in Python.

7.2 Correctness and traps

Sorting places every price $\leq x$ before every price $> x$; the first position after the former group is therefore exactly their count. Preprocessing costs $O(n \log n)$ and each query $O(\log n)$. Test a budget below every price, equal to duplicate prices, and above every price. `lower_bound` counts $< x$ and is wrong at equality.

8 7. 520B – Two Buttons

Official: <https://codeforces.com/problemset/problem/520/B>

8.1 State graph

An integer is a vertex, and each button press is a unit-cost directed edge. BFS finds the fewest presses once the state space is bounded. If $n \geq m$, only decrementing is useful, so answer $n-m$. Otherwise no optimal route needs to visit a value above $2m$: after first exceeding m , a decrement-only suffix reaches m ; overshooting past $2m$ cannot beat stopping at or below $2m$.

8.2 Algorithm and proof

Run BFS on $[0, 2m]$ from n , visiting $x-1$ when nonnegative and $2x$ when within the bound. BFS explores paths by increasing length, so its first visit to m is optimal. The bound argument ensures an optimal path is represented. Time and memory are $O(m)$ in the $n < m$ branch.

Test $n=m$, $n > m$, $m=1$, and a case where doubling first is worse than a decrement. Mark visited when enqueueing, not when dequeuing.

9 8. 455A – Boredom

Official: <https://codeforces.com/problemset/problem/455/A>

9.1 Compress the story by value

Choosing value x eventually earns $\text{gain}[x] = x * \text{frequency}[x]$; taking all copies is always at least as good as taking fewer. It forbids only neighboring values. Thus $\text{dp}[x]$ is the best score using values at most x .

```
dp[x] = max(dp[x-1], dp[x-2] + gain[x])
```

9.2 Correctness and implementation

An optimal solution either omits x , giving $\text{dp}[x-1]$, or takes it, forcing the rest to use values at most $x-2$, giving the second term. These exhaustive, disjoint cases prove the recurrence by induction. Sweep through the maximum input value using two rolling `long long` states. Complexity is $O(\text{max value} + n)$ time and $O(\text{max value})$ memory for frequencies.

Test repeated equal values, values separated by gaps, and a choice where one large frequency beats a neighboring value. Do not sort individual occurrences and apply the recurrence to positions; the conflict is between values.

10 9. 300C – Beautiful Numbers

Official: <https://codeforces.com/problemset/problem/300/C>

10.1 Count by number of a digits

If exactly i positions contain digit a , the digit sum is $s = i*a + (n-i)*b$, and there are $C(n,i)$ strings with that composition. Check whether every decimal digit of s is either a or b ; if so, add the binomial coefficient.

10.2 Proof and complexity

Every legal string has one unique i , and all strings with that i have the same digit sum. The digit check selects exactly the legal compositions, while the binomial coefficient counts their distinct position choices. Precompute factorials and inverse factorials modulo the prime, using Fermat's inverse. The work is $O(n + \text{number of digits checked across } n+1 \text{ sums})$, effectively $O(n \log n)$.

The input guarantees $1 \leq a < b \leq 9$, so the digit sum is positive. Test $a=1$, $b=9$, a sum containing a forbidden interior digit, and a composition where only one value of i succeeds. Use modular arithmetic only for counts, never for the decimal digit sum.

11 10. 166B – Polygons

Official: <https://codeforces.com/problemset/problem/166/B>

11.1 Reduce containment to convex point location

Because A is convex, if every vertex of B is strictly inside A , every edge and every interior point of B is also inside A : they are convex combinations of points of A 's interior. Thus test every B vertex against A . For one point P , use $A[0]$ as a fan pivot. First reject P unless it is strictly between rays $A[0] \rightarrow A[1]$ and $A[0] \rightarrow A[n-1]$; then binary-search the fan sector $(A[0], A[i], A[i+1])$ containing P and require strict inclusion in that triangle using cross-product signs.

11.2 Why this is sufficient

A convex polygon is partitioned by its pivot fan into triangles. The ray tests select a unique possible sector, and the triangle test is equivalent to strict membership in that sector. Therefore every accepted B vertex is strictly in A . Convexity then proves the whole of B is inside; any rejected vertex proves it is not. The time is $O(m \log n)$ and memory is $O(1)$ beyond the polygons.

Use `long long` for cross products. Test clockwise input, an inner vertex on A 's edge, a B edge that comes close to A , and a point just beyond one fan boundary. Never use floating point.

12 11. 20C – Dijkstra?

Official: <https://codeforces.com/problemset/problem/20/C>

12.1 Model and algorithm

All edge weights are positive, so Dijkstra applies. Store `dist[v]` and `parent[v]`; on a strict relaxation of $u \rightarrow v$, set `parent[v]=u`. Use a min-heap of `(distance, vertex)` pairs and ignore stale entries. If `n` remains infinite, print `-1`; otherwise follow parents from `n` to 1 and reverse.

12.2 Correctness proof

When Dijkstra extracts a non-stale vertex, no unexplored path can improve it: such a path first crosses from an extracted prefix through a nonnegative edge, so it cannot arrive cheaper. Thus every relaxation uses a correct shortest prefix. A parent is assigned only when an edge creates the current best distance, so following parents reconstructs a shortest path to vertex 1. Complexity is $O((n+m) \log n)$.

Test disconnected graphs, parallel edges, and a route whose first discovered path is later improved. Distances need `long long`; a boolean `visited` on heap insertion would incorrectly freeze an improvable vertex.

13 12. 161D – Distance in Tree

Official: <https://codeforces.com/problemset/problem/161/D>

13.1 Bounded-depth tree DP

Root the tree. Let $\text{cnt}[u][d]$ be the number of vertices in u 's subtree at distance d from u , for $0 \leq d \leq k$; initialize $\text{cnt}[u][0]=1$. Process a child v after its table is ready. Before merging it, every pair with one end in the accumulated part of u and the other in v contributes when $d_1 + d_2 + 1 = k$.

13.2 Merge, proof, and complexity

For each child and each $0 \leq d < k$, add $\text{cnt}[v][d] * \text{cnt}[u][k-d-1]$ to the answer, then shift $\text{cnt}[v][d]$ into $\text{cnt}[u][d+1]$. Every pair has a unique lowest common ancestor; it is counted exactly when the second relevant child subtree merges there. No pair in one child is recounted because that child already handled it. The bounded $k \leq 500$ gives $O(nk)$ time and memory.

Test a path, a star, $k=1$, and k larger than the diameter. Count in `long long`; merge cross-pairs before shifting the child table.

14 13. 580D – Kefa and Dishes

Official: <https://codeforces.com/problemset/problem/580/D>

14.1 State captures the only relevant history

Use `dp[mask][last]`: maximum satisfaction for exactly the dishes in `mask` ordered with `last` final. Append an unused dish `next`:

```
dp[mask | bit(next)][next] = max(current,
                                   dp[mask][last] + base[next] + bonus[last][next])
```

Start each one-dish mask with its base value, and maximize states with `popcount(mask)=m`.

14.2 Proof and costs

The future bonus depends only on the final dish and the used set; earlier order details have already contributed their adjacent bonuses. Inductively, each transition appends every possible final dish to every valid shorter ordering, so the table considers every legal ordered selection exactly by its best value. There are $2^n * n$ states and $O(2^n * n^2)$ transitions.

Test asymmetric bonuses, `m=1`, and a bonus that makes a lower-base dish worth choosing. The common wrong state is `dp[mask]`: it loses the predecessor needed to price the next append.

15 14. 1245D – Shichikuji and Power Grid

Official: <https://codeforces.com/problemset/problem/1245/D>

15.1 Virtual-source reduction

Create vertex 0. Edge $(0, i)$ means build a station in city i , with the given station cost. Edge (i, j) means wire the two cities, with the supplied Manhattan-based cost. Any plan that powers all cities corresponds to a connected subgraph containing 0, and vice versa. The minimum plan is therefore an MST.

15.2 Algorithm and proof

Generate all $O(n^2)$ city edges plus source edges and run Kruskal, recording source edges as stations and the others as wires. An MST is no heavier than any connected subgraph because removing cycles never disconnects it or increases cost. The correspondence proves its output is an optimal power plan. With $n \leq 2000$, $O(n^2 \log n)$ is acceptable.

Use 64-bit costs. Test that a very expensive station is replaced by a wire, that two cheap stations remain separate, and that positive coordinates have a large Manhattan difference.

16 15. 271D – Good Substrings

Official: <https://codeforces.com/problemset/problem/271/D>

16.1 Enumerate starts, identify contents exactly

The length limit permits $O(n^2)$ substring enumeration. For each start, extend the end while maintaining the bad-character count; stop once it exceeds k . Insert the traversed characters into a trie. A trie node represents one unique prefix from the root and hence one unique substring; creating a node counts a new distinct valid substring. With at most $n(n+1)/2$ nodes and 26 lowercase transitions per node, an `array<int,26>` trie fits the stated 512 MB limit in C++ at $n=1500$.

16.2 Correctness and testing

The maintained bad count is exactly the count in the current extension, so the loop inserts every and only valid occurrences. Equal contents follow the same trie edges and end at the same node; distinct contents diverge at their first different character and end at different nodes. Thus the created-node count is exactly the desired number. Work is $O(n^2)$ time and $O(n^2 * \text{alphabet})$ integer storage in the fixed-alphabet implementation.

Test repeated characters, all-good/all-bad masks, $k=0$, and repeated equal substrings from different starts. Do not insert `(start,end)`: that counts occurrences, not distinct strings. A double hash is a smaller-memory optional Python strategy, but it is probabilistic and is not the proof-backed default.

17 16. 61E – Enemy Is Weak

Official: <https://codeforces.com/problemset/problem/61/E>

17.1 Split each decreasing triple at its middle

For each j , multiply the number of earlier values greater than $a[j]$ by the number of later values smaller than $a[j]$. Sum the products. Coordinate compress values; a Fenwick tree scans left-to-right for earlier counts, and a second scan right-to-left obtains later counts.

17.2 Proof and complexity

Every valid triple has one unique middle index j , and choosing either valid endpoint is independent once $a[j]$ is fixed. Thus the product counts exactly the triples centered at j , with no overlap between different middles. Fenwick updates and prefix queries make the time $O(n \log n)$ and memory $O(n)$.

For duplicates, strict comparisons require `seen - prefix(rank)` and `prefix(rank-1)`; this task gives distinct values, but writing the strict forms prevents a bad habit. Test sorted increasing, sorted decreasing, and $n < 3$.

18 17. 86D – Powerful Array

Official: <https://codeforces.com/problemset/problem/86/D>

18.1 Maintain the statistic while moving a range

Order queries by Mo blocks. Keep `freq[x]` and current answer `sum x * freq[x]^2`. Adding value `x` changes its contribution from `x*f^2` to `x*(f+1)^2`; remove the old contribution, increment, and add the new one. Removal is the reverse. Move `L` and `R` one step at a time to each next query, then store the current answer at its original query index.

18.2 Why it works

The invariant is exact: `freq` represents precisely the current interval and the answer is the sum of each value's contribution from those frequencies. Each primitive move preserves it by replacing only the changed value's term. Mo ordering changes visit order, not query contents, so each saved answer is correct. Typical complexity is $O((n+q) \sqrt{n})$ moves with $O(n)$ memory.

Use `long long`. Test repeated one value, moving both endpoints backward, and queries returned in a different order than their sorted processing order.

19 18. 498C – Array and Operations

Official: <https://codeforces.com/problemset/problem/498/C>

19.1 Decompose operations by prime

Factor every number. Fix a prime p : each exponent at an odd-index vertex is supply, each exponent at an even-index vertex is demand, and each allowed odd-even pair is an infinite-capacity edge. The maximum number of p -units removed is a max flow. Sum independently over all primes that occur.

19.2 Reduction proof

Any allowed division by a composite can be split into one operation per prime factor on the same pair. This preserves feasibility and increases (or at least preserves) the number of counted operations, so some optimum uses prime-only operations. A flow unit travels from an odd exponent through an allowed pair to an even exponent, exactly representing one removable common prime unit. Capacities prevent using an exponent unit twice. Conversely every sequence of prime-unit operations yields such a flow. Therefore max flow equals the optimum count for that prime, and primes do not interact. Factorization plus Dinic per occurring prime is feasible for $n, m \leq 100$.

Normalize each pair orientation by index parity. Test no shared prime, one large repeated prime exponent, and a path where flow must choose between two neighbors.

20 19. 319C – Kalila and Dimna in the Logging Industry

Official: <https://codeforces.com/problemset/problem/319/C>

20.1 Derive lines before choosing a hull

The recurrence has form

$$dp[i] = \min_{j < i} (dp[j] + b[j] * a[i])$$

For fixed j , this is the line $y = b[j] * x + dp[j]$ queried at $x=a[i]$. The base case is $dp[1]=0$: insert line $y=b[1]x$ before querying for $i=2$. After computing $dp[i]$, insert $y=b[i]x+dp[i]$. The stated monotonicity gives query $a[i]$ values increasing and slopes $b[j]$ decreasing, so a monotone lower hull with a moving query pointer works; a Li Chao tree is a robust alternative.

20.2 Hull invariant and proof

Keep candidate lines in slope order. When a new line makes the intersection of the last two candidates lie no later than the prior intersection, the middle line is never minimal for any future increasing query and can be removed. The front pointer advances only while its successor is no worse at the current query. Thus the queried line is minimum among all inserted lines, which equals the recurrence minimum by induction. The monotone hull runs in $O(n)$ time; Li Chao gives $O(n \log \text{coordinate range})$.

Use cross multiplication with wide integers for redundancy tests, avoiding floating point. Test equal-looking line values at a query, a line optimal only for a short interval, and values near 10^9 .

21 Stress and carry-forward

For Gates 1–2, compare random small cases to brute force: enumerate windows, paths, subsets, or point classifications. For Gates 3–4, first validate the model on tiny instances: exhaustive dish orders, a brute MST, exact substring sets, all triples, a direct range score, and a small flow implementation. The reusable lesson is not the task name. It is the chain:

```
statement signal -> state/invariant -> proof obligation -> update rule -> test
```